

Predicting NBA Playoff Underperformance

A game-level ML system with live API and web app

PRESENTED BY
Hanzhang Yuan

DATE
June 2, 2026

LIVE DEPLOYED SERVICES

Web App (Cloud Run primary)
<https://nba-playoff-app-803317660037.us-central1.run.app>

Web App (Streamlit backup)
<https://stat418-final-project-stnrtix9jwpgzt5m6cfhfy.streamlit.app>

REST API + Swagger Docs
<https://nba-playoff-api-803317660037.us-central1.run.app/docs>

Why Some Stars Fade in the Playoffs

THE PROBLEM

Can we predict, before tip-off, whether an NBA player will underperform his regular-season Game Score baseline in a specific playoff game?

WHY IT MATTERS

Playoff basketball is a different game.

Playoff basketball changes the decision environment: defenses scout more aggressively, rotations often tighten, and every matchup gets repeated pressure. A pre-game underperformance model gives fans and analysts a structured way to quantify fade risk rather than relying only on narratives.

KEY OBJECTIVES

1. Scrape 22 seasons of NBA data ·
2. Build leakage-safe player-game features ·
3. Compare models and explain predictions ·
4. Deploy a usable App + API

PROBLEM TYPE

Binary

Relative underperform
yes / no

UNIT OF ANALYSIS

Player-game

Not player-season

DATA RANGE

22 seasons

2003-04 to 2024-25

Scraping 22 Seasons of NBA Box Scores

DATA SOURCE

NBA Stats API via nba_api

NBA Stats API via nba_api: player game logs (regular + playoff) and team advanced stats from 2003-04 through 2024-25.

PREPROCESSING PIPELINE

- | | |
|--------------|---|
| 1. Scrape | 1.2s rate limit + 5-retry backoff + disk cache (resumable) |
| 2. Parse | JSON → Parquet with Hollinger Game Score computed |
| 3. Aggregate | Per-season per-player + per-team-defense stats |
| 4. Engineer | 45 features: per-season z-scores + expanding-window playoff history |

REQUESTS

~25,000
across 22 seasons

PLAYER-GAMES

575,801
regular + playoff

MODELING SET

~30,000
filtered playoff rows

CHALLENGES SOLVED

Era drift → per-season z-score | Connection resets → patched headers + retry/backoff | Noisy baselines → relative 20% threshold + 1.0 floor

Three Models, 45 Features, Strict Time Split

MODELS COMPARED (held-out 2023-25 test set; all AUC ≈ 0.65)

BASLINE

Logistic Regression

Test AUC: 0.650

F1: 0.638

MID-TIER

Random Forest

Test AUC: 0.650

F1: 0.621

DEPLOYED

XGBoost

Test AUC: 0.649

F1: 0.632

Chosen for SHAP explanations + robust serving

TOP SHAP FEATURES



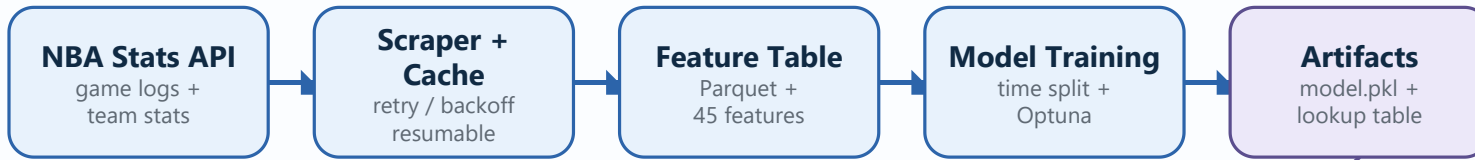
45 FEATURES · 6 FAMILIES

- A. Player regular-season baseline (15)
- B. Player playoff history (10)
- C. Game context (8)
- D. Opponent strength (5)
- E. Player × opponent matchup (4)
- F. In-series momentum (3)

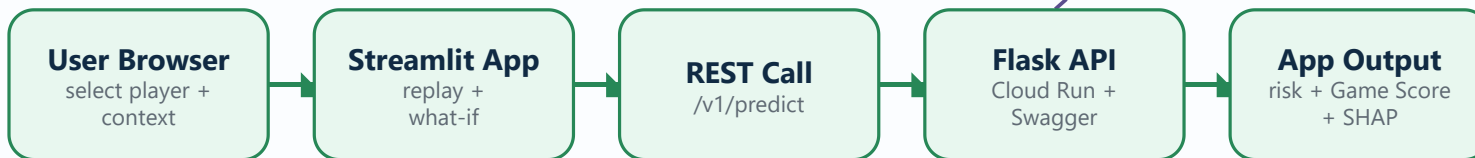
Train 2003-2020 Val 2021-23 Test 2023-25 · Key finding: target definition (+3.8 pp AUC) mattered more than model choice (~0.1 pp)

Offline Training + Online Inference Architecture

OFFLINE TRAINING FLOW



ONLINE INFERENCE FLOW



Offline: NBA Stats API → cached data → feature table → model artifacts

Online: Streamlit inputs → Cloud Run API → risk + Game Score + SHAP outputs

TECH STACK & DEPLOYMENT

Data	nba_api · pandas · pyarrow
Model	scikit-learn · XGBoost · Optuna · SHAP
API	Flask · flask-restx · gunicorn
App	Streamlit Community Cloud
Deploy	Docker · Google Cloud Run
Quality	pytest · GitHub Actions

Two live services
Streamlit UI + Cloud Run API

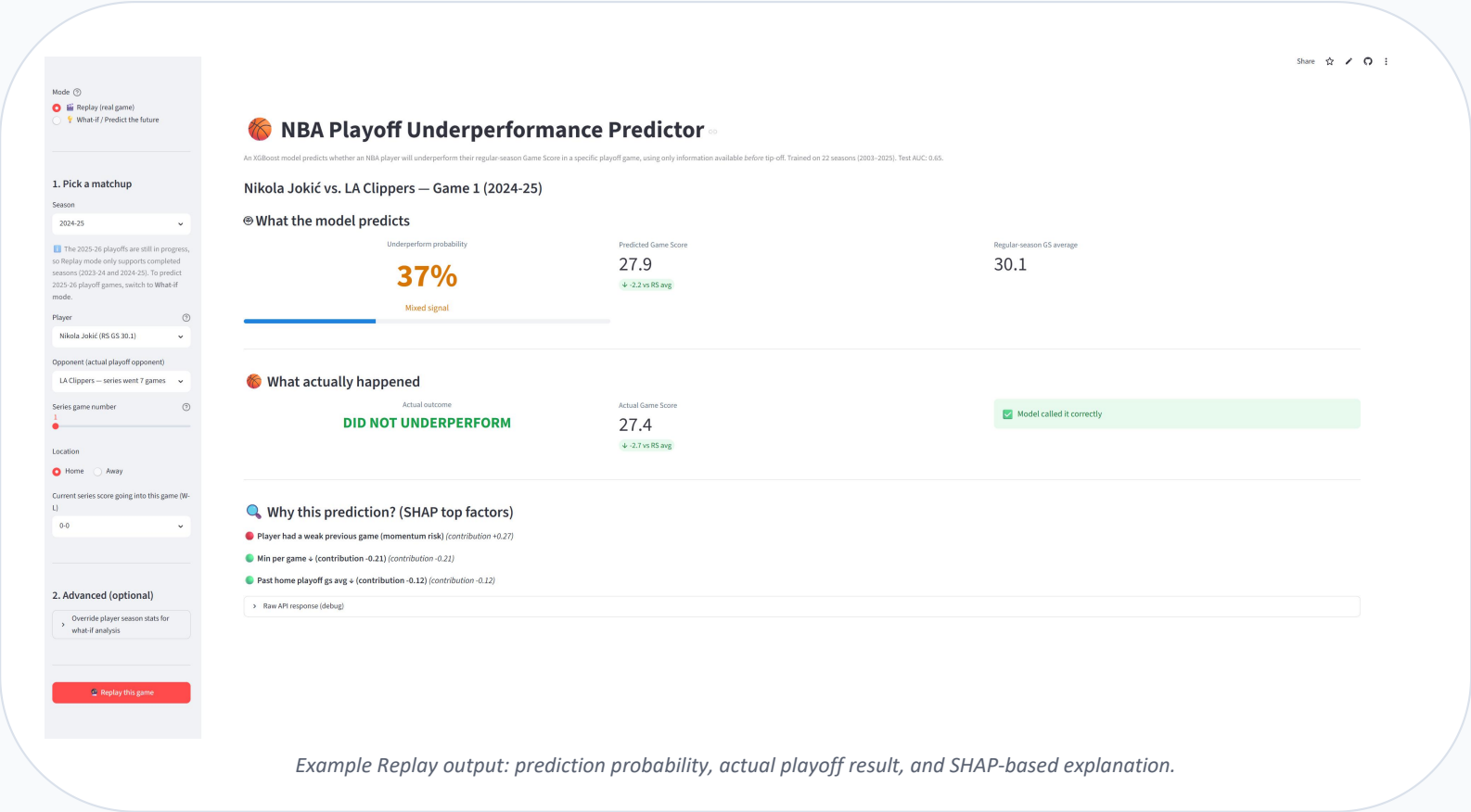
Live App Walkthrough · Replay Mode + What-if Mode

REPLAY MODE

- Real game · model vs actual result
- Select season, player, opponent, game
 - Predicts before seeing actual outcome
 - Compares prediction with Game Score

WHAT-IF MODE

- Future/hypothetical prediction
- Choose player, opponent, home/away
 - Adjust series context and overrides
 - Returns probability and GS forecast



Screenshot from deployed Streamlit app: prediction probability, actual result, and SHAP explanation.

Challenges, AI Assistants, Lessons, Next Steps

TOP TECHNICAL CHALLENGES

Data leakage in expanding-window features

Wrote 7 pytest assertions BEFORE the pipeline → caught 2 real bugs

nba_api connection resets from corporate network

Patched request headers + retry/backoff + disk cache

Heteroskedastic Game Score baselines

Relative 20% threshold + 1.0 GS floor → +3.8 pp test AUC

API returning identical predictions for all inputs

Recompute series-momentum from real games 1..N-1 at lookup time

AI ASSISTANT ROLE

Used AI as a development assistant for boilerplate code, debugging guidance, UI layout, and deployment review. Core decisions—including feature engineering, target definition, model evaluation, and final verification—were made and tested by me.

KEY LESSON: A clean target definition and leakage control mattered more than choosing a complex model.

FUTURE WORK: scheduled weekly scrape · injury features · series-level model · stronger calibration · 2025-26 in-progress predictions